

Designing and Analysis of Algorithms

Course Code: ECS 5101/CS514

- Dr Rahul Mishra
- IIT Patna

- **Lecture 16**

m-Way Search Trees :-

- > All the data structures discussed so far favor data stored in the internal memory and hence support internal information retrieval.
- > To favor retrieval and manipulation of data stored in external memory, viz. **storage devices such as Hard disk**. It is required to have some specialized data structure.
- > 'm-way search tree', B-tree, and B⁺ tree are examples of such data structures.
- > m-way search trees are generalized versions of binary search trees. The goal of m-way search tree is to minimize the access while retrieving a key from a file.
- > An m-way search tree of height h calls for $O(h)$ number of access for an **insert/delete/retrieval** operation.
- > It pays to ensure that the height h is close to $\log_m(n+1)$.
- > It is because the number of elements in an m-way search tree of height h ranges from a minimum of h to maximum of $m^h - 1$.

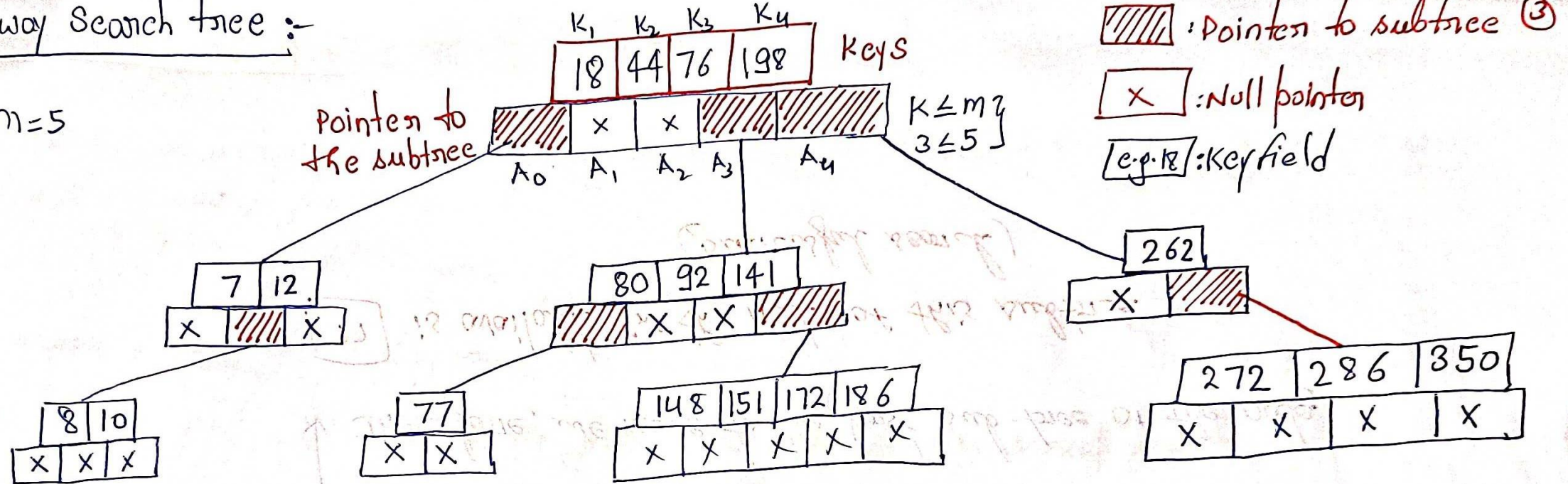
Definition:

An m -way search tree T may be an empty tree. If T is not empty, it satisfies the following properties:

- (i) For some integer m , known as order of the tree, each node is of degree which can reach a maximum of m , in other words, each node has, at most m child nodes.
A node may be represented as $A_0, (K_1, A_1), (K_2, A_2), \dots, (K_{m-1}, A_{m-1})$ where $K_i, 1 \leq i \leq m-1$ are the keys and $A_i, 0 \leq i \leq m-1$ are the pointers to subtrees of T .
- (ii) If a node has K child nodes where $K \leq m$, then the node can have only $(K-1)$ keys $K_1, K_2, \dots, K_{K-1}$, contained in the node such that $K_i < K_{i+1}$ and each of the keys partitions all the keys in the subtrees in K subsets.
- (iii) For a node $A_0, (K_1, A_1), (K_2, A_2), \dots, (K_{m-1}, A_{m-1})$, all key values in the subtree pointed to by A_i are less than the key $K_{i+1}, 0 \leq i \leq m-2$ and all key values in the subtree pointed to by A_{m-1} are greater than K_{m-1} .
- (iv) Each of the subtrees $A_i, 0 \leq i \leq m-1$ are also m -way search trees.

5-way search tree :-

$m=5$



① Each node of degree ≤ 5 / Each node is having at max 5 child

② $A_i \in 0 \leq i \leq m-1$ and $K_i \in 1 \leq i \leq m-1$

③ $K_i < K_{i+1}$

④ A_i (pointing keys) $< K_{i+1}$

⑤ $A_i \rightarrow$ sub-tree are m -way search trees.

* Searching an m-Way Search Tree:

(4)

- > Searching for a key in an m-way search tree is similar to that of binary search trees.
- > e.g., search **77** in the above tree;
- > Step:1 > **77 > 76 > 44 > 18**

↓ Move to the fourth sub-tree

77 < 80

↓ Therefore, we move to the first sub-tree of the node;

77 is available in the node of this subtree.
(successful search)

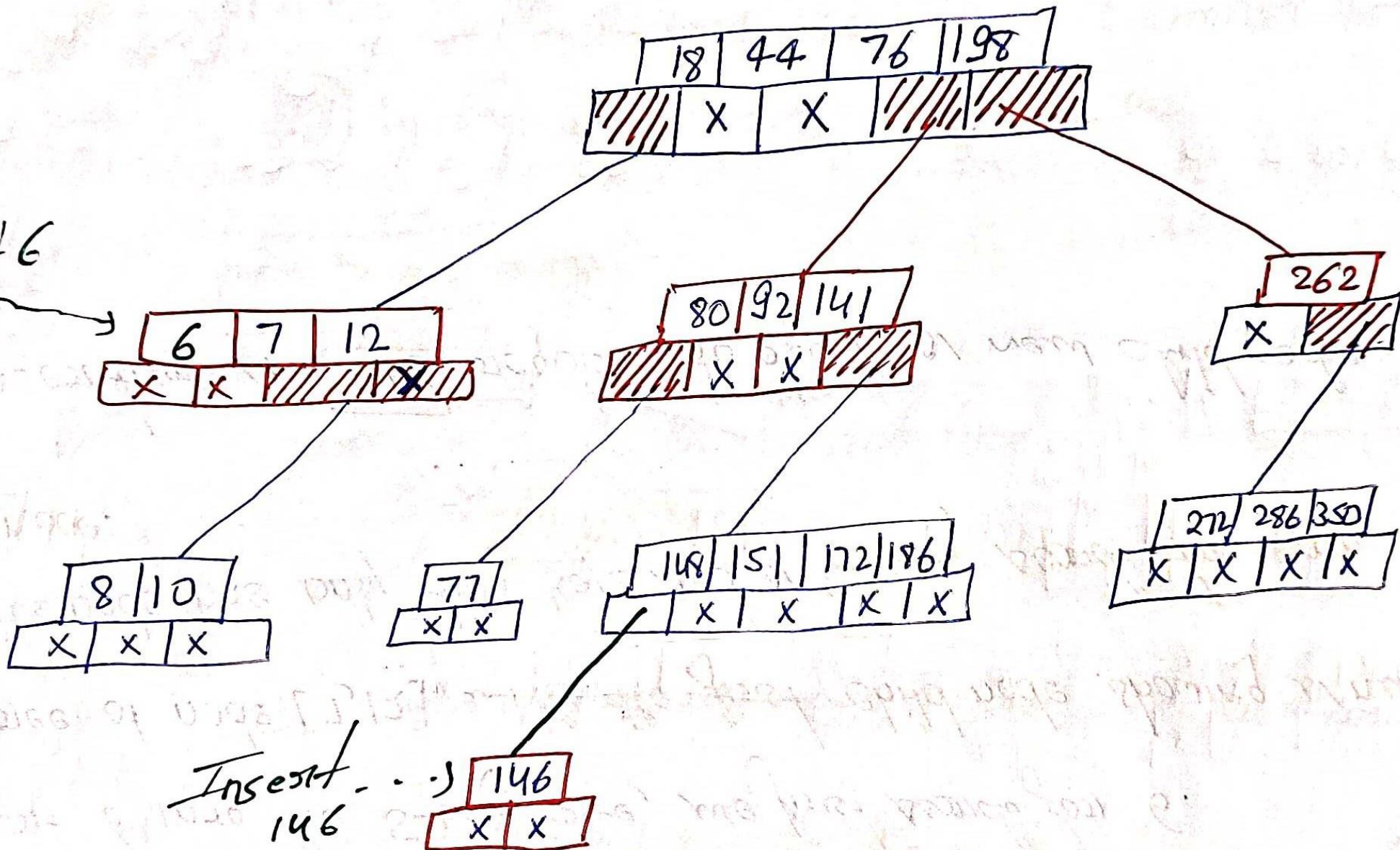
Insertion in an m-way Search Tree :-

⇒ To insert a new element into an m-way search tree we proceed in the same way as one would in order to search for the element.

⇒ For example :-

- To insert 6 into the 5-way tree, we first search for 6.
- The tree of node $[7, 12]$, with the first child node showing a null pointer.
- Since the node has only two keys and a 5-way search tree can accommodate upto 4 keys in a node.
- But insertion of $[14, 6]$ requires to open a new child node.

Insert 6



Designing and Analysis of Algorithms

Course Code: ECS 5101/CS514

- Dr Rahul Mishra
- IIT Patna

- **Lecture 17**

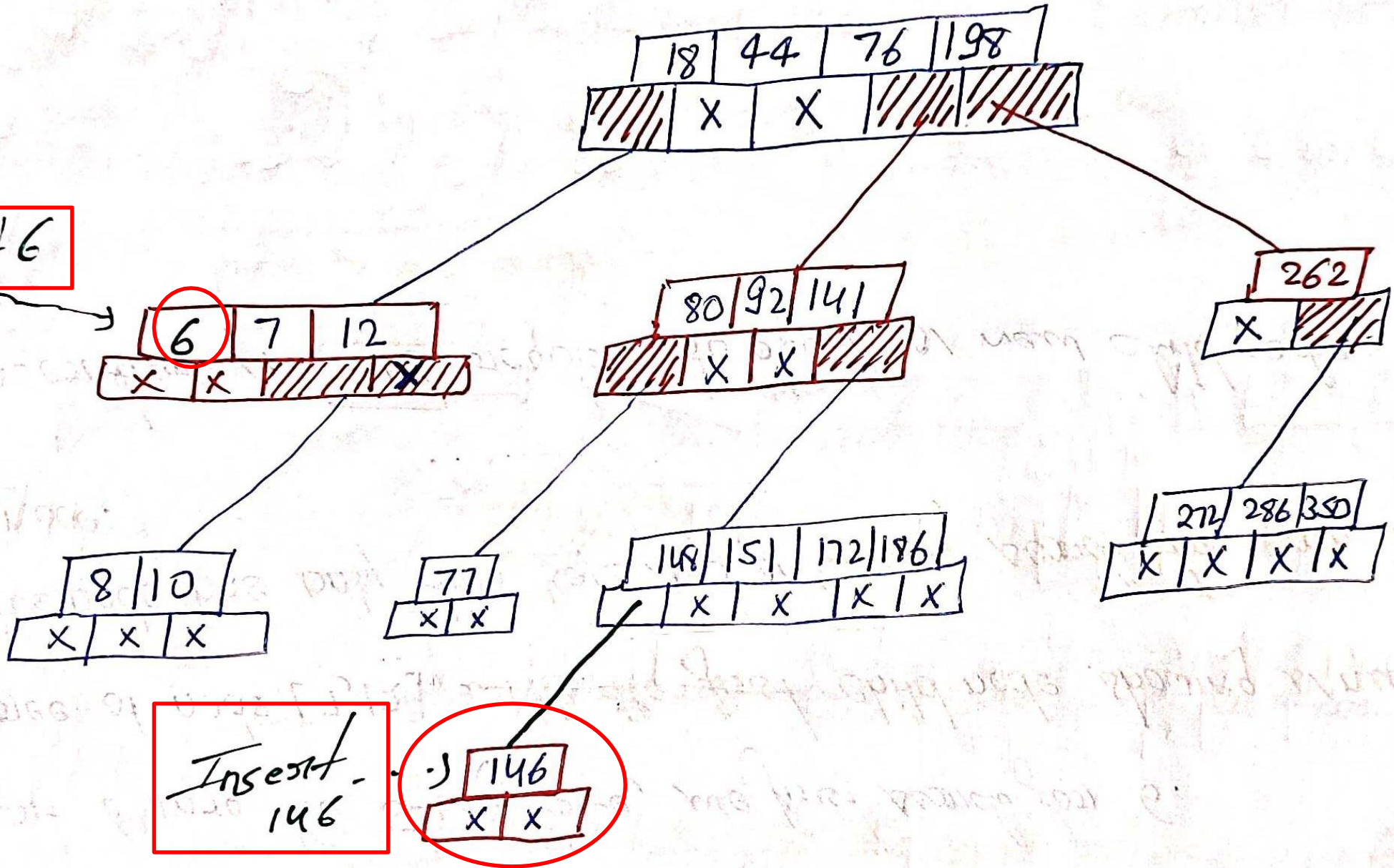
Insertion in an m-way Search Tree :-

⇒ To insert a new element into an m-way search tree we proceed in the same way as one would in order to search for the element.

⇒ For example :-

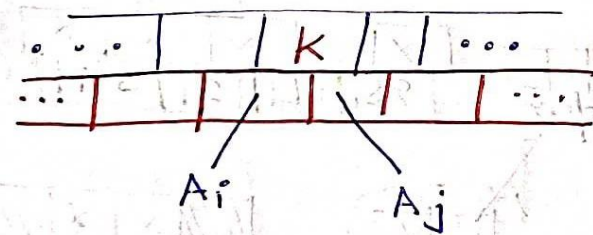
- To insert 6 into the 5-way tree, we first search for 6.
- The tree of node $[7, 12]$, with the first child node showing a null pointer.
- Since the node has only two keys and a 5-way search tree can accommodate up to 4 keys in a node.
- But insertion of $[14, 6]$ requires to open a new child node.

Insert 6



Deletion in an m-way Search Tree →

- Let K be a key to be deleted from the m-way search tree.
- To delete the key we proceed as one would to search for the key.



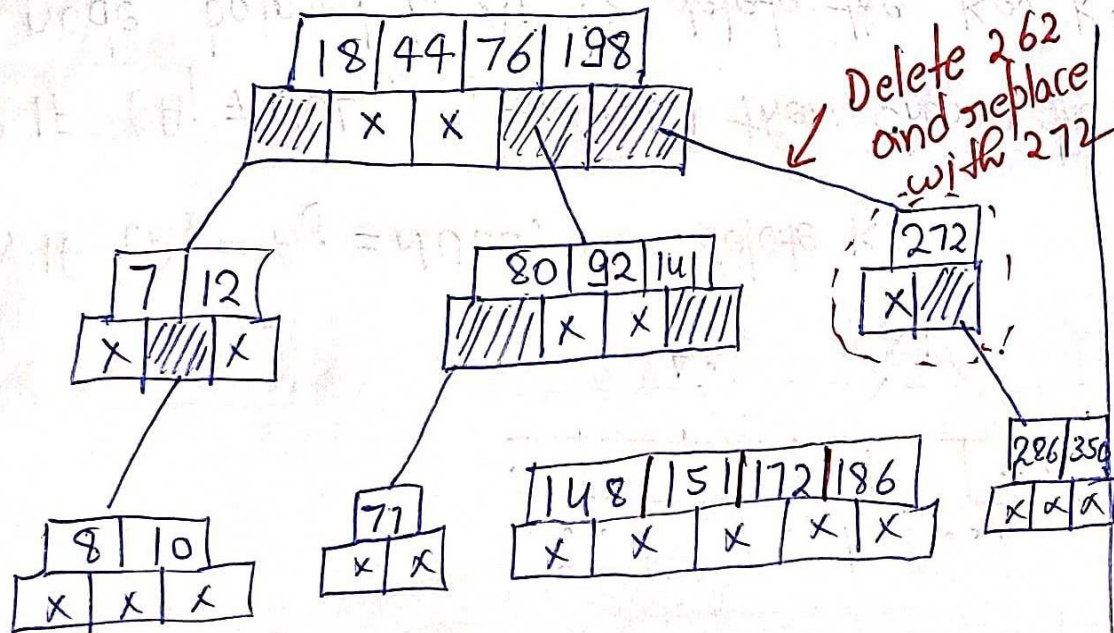
K : key
 A_i, A_j : Pointers to subtrees

- 1) IF $(A_i = A_j = \text{NULL})$ then delete K
- 2) IF $(A_i \neq \text{NULL}, A_j = \text{NULL})$ then choose the largest of the key element K' in the child node pointed by A_i
 - i) delete the key K'
 - ii) Replace K by K'
 - iii) Obviously deletion of K' may call for subsequent replacements and therefore deletions in a similar manner.

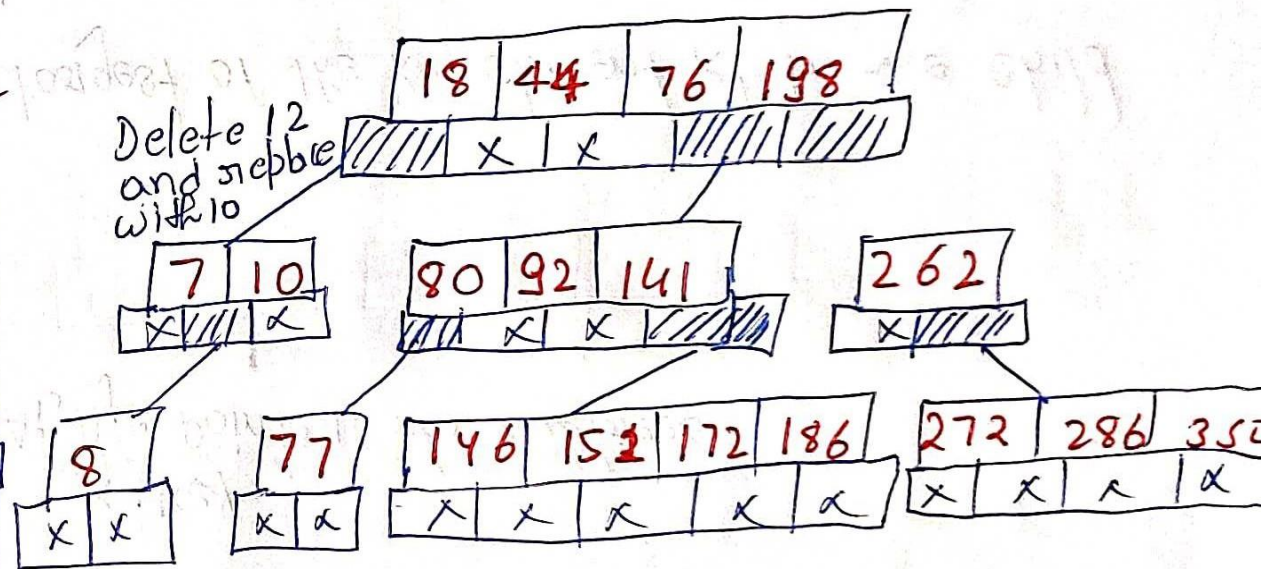
3) IF ($A_i = \text{NULL}$, $A_j \neq \text{NULL}$) then choose the smallest of the key element (k'') from the subtree pointed to by A_j delete k'' and replace k by k'' .

Again there may be some subsequent calls.

4) IF ($A_i \neq \text{NULL}$ and $A_j \neq \text{NULL}$) then choose either of 3 or 4.



(a)



(b)

B-Trees :

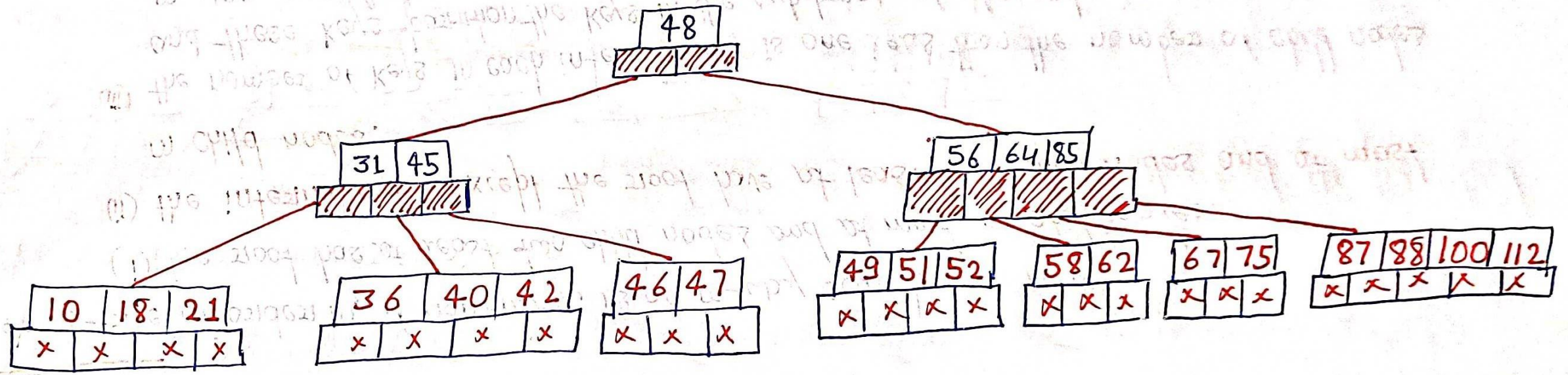
- > m-way search trees have the advantage of minimizing file access due to their restricted height.
- > However, it is essential that the height of the tree be kept as low as possible and therefore there arises the need to maintain balanced m-way trees. Such a balanced m-way search tree is what is defined as B-tree.

Definition →

A B-tree of order m , if non empty, is an m-way search tree in which:

- (i) the root has at least two child nodes and at most m child nodes.
- (ii) the internal nodes except the root have at least $\lceil \frac{m}{2} \rceil$ child nodes and at most m child nodes.
- (iii) the number of keys in each internal node is one less than the number of child nodes and these keys partition the keys in the subtrees of the node in a manner similar to that of m-way search trees.
- (iv) all the leaf nodes are on the same level.

* A B-tree of order 3 is referred to as 2-3 tree since the internal nodes are of degree 2 or 3 only.



Pointers to subtrees

Key fields

Null pointers

Searching a B-tree →

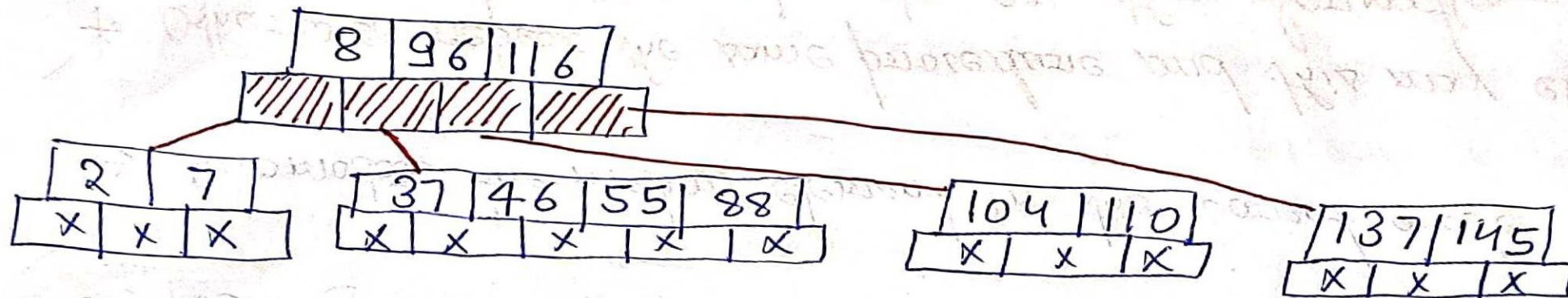
→ Searching for a key in a B-tree is similar to have an m-way search tree.

→ The number of access depends on the height h of the B-tree.

Insertion in a B-tree:

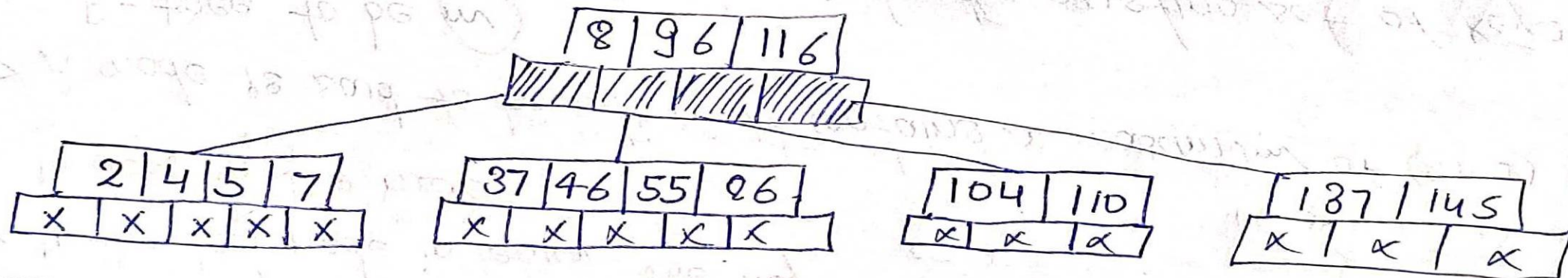
- > The insertion of a key in a B-tree proceeds as if one were searching for the key in the tree.
- > If the leaf node in which the key is to be inserted is not full, then the insertion is done in the node.
- > A node is said to be full if it contains a maximum of $(m-1)$ keys, given the order of B-tree to be m .
 - ↳ Insert the key in order into the existing set of keys in the node, split the node at its median into two nodes at the same level.
 - ↳ pushing the median element up by one level
 - ↳ Accomodate the median element in the parent node if it is not full
- * Otherwise repeat the same procedure and this may even call for rearrangement of the keys in the root node on the formation of a new root itself.

Consider the B-tree ÷

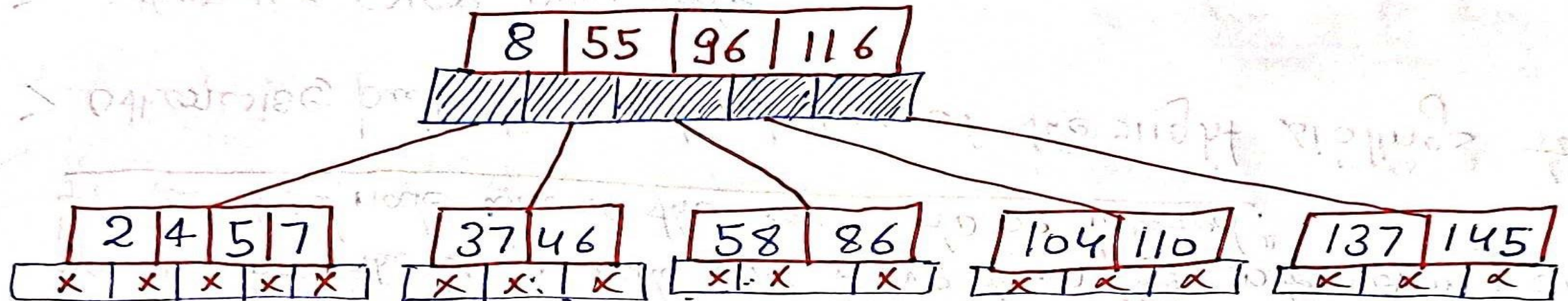


a) Insert the element 4, 5, 58, 6

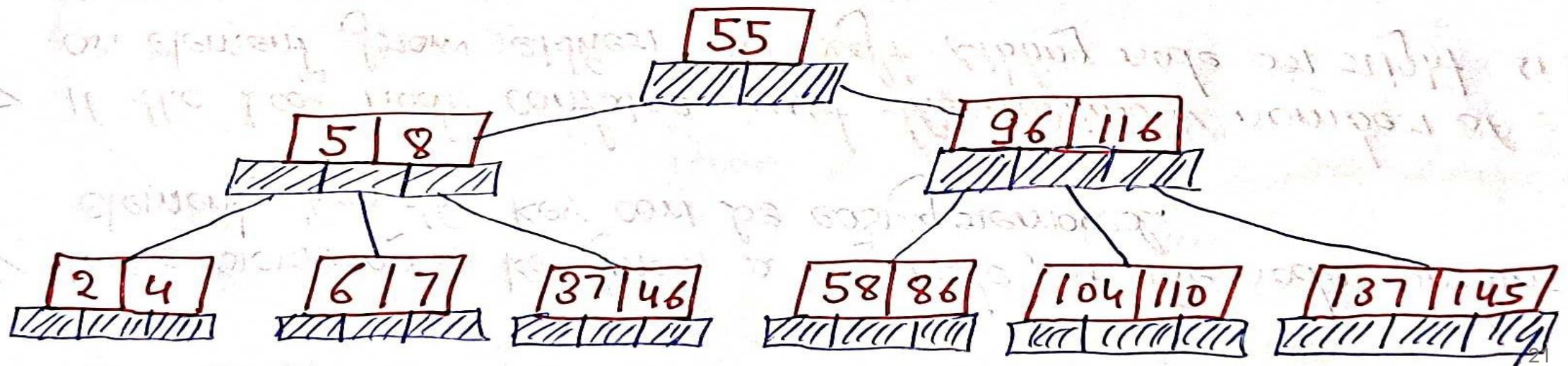
b) Insert 4, 5 (simpler)



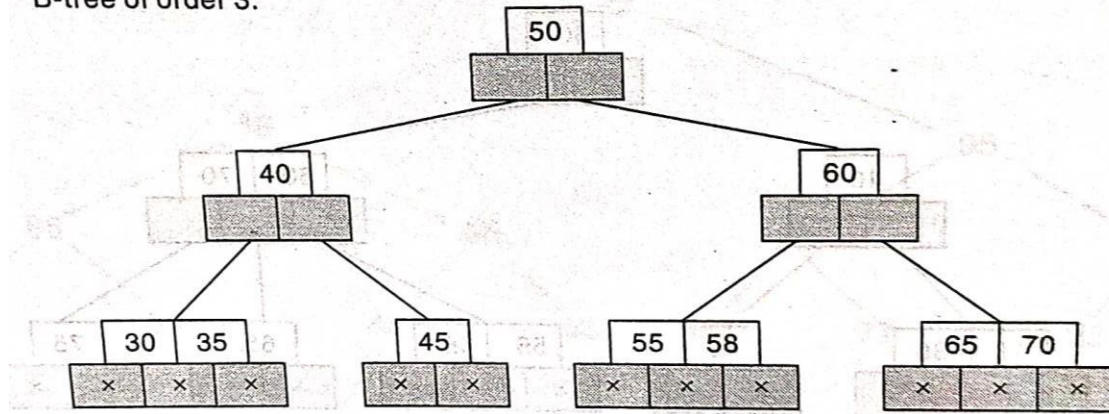
c) Insert 58



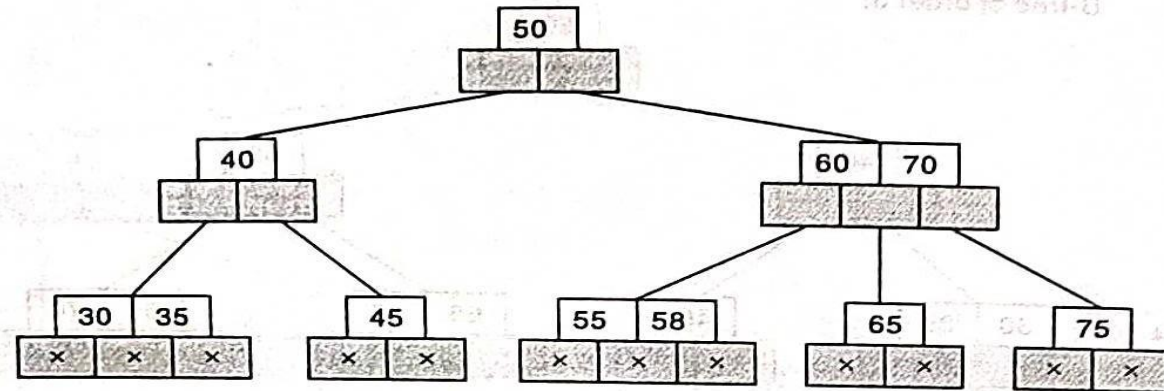
d) Insert 6:



B-tree of order 3:



Insert 75:



Insert 57:

